

```

syntax = "proto3";

import "google/protobuf/timestamp.proto";
import "google/protobuf/duration.proto";

option csharp_namespace = "JLokaGrpcService.Protos";

message CreateAddressRequest {
    string first_name = 1;
    string last_name = 2;
    string email = 3;
    string phone = 4;
    int32 year_of_birth = 5;
    bool is_active = 6;
}

message UpdateInvoiceDueDateRequest {
    string invoice_id = 1;
    google.protobuf.Timestamp due_date = 2;
    google.protobuf.Duration grace_period = 3;
}

*****

enum InvoiceStatus {
    INVOICE_STATUS_UNKNOWN = 0;
    INVOICE_STATUS_DRAFT = 1;
    INVOICE_STATUS_AWAIT_PAYMENT = 2;
    INVOICE_STATUS_PAID = 3;
    INVOICE_STATUS_OVERDUE = 4;
    INVOICE_STATUS_CANCELLED = 5;
}

*****

```

Every enum type **must contain a 0 value**, which is the default value and must be placed at the first position.

Protobuf scalar value types, such as int32, sint32, fixed32, and bool, **cannot be null**. But in .NET, **nullable value types are very common**. For example, we can use int? to declare an integer value that can be null. To support nullable value types, protobuf provides some wrapper types, which are defined in the [google/protobuf/wrappers.proto](#) file, to support nullable types.

Most of .NET nullable types are supported by protobuf. Besides the `google.protobuf.DoubleValue`, `google.protobuf.Int32Value`, and `google.protobuf.BoolValue` types, protobuf also provides the following wrapper types:

- `google.protobuf.FloatValue`: This type is used to represent a `float?` value.
- `google.protobuf.Int64Value`: This type is used to represent a `long?` value.
- `google.protobuf.UInt32Value`: This type is used to represent a `uint?` value.
- `google.protobuf.UInt64Value`: This type is used to represent a `ulong?` value.
- `google.protobuf.StringValue`: This type is used to represent a `string` value.
- `google.protobuf.BytesValue`: This type is used to represent a `ByteString` value.

There are two special types in the preceding list: `google.protobuf.StringValue` and `google.protobuf.BytesValue`. The corresponding .NET types are `string` and `ByteString`. The `ByteString` type is a class that represents an immutable array of bytes, which is defined in the protobuf runtime. The default value of these two types is `null`.

Default values

The following table lists the default values of the scalar value types:

Protobuf Type	Default Value
<code>string</code>	An empty string
<code>bytes</code>	An empty byte array
<code>bool</code>	<code>false</code>
Numeric types	0
enums	The first enum value

Table 11.2 – Default values of protobuf scalar value types

gRPC supports four types of RPC methods:

- **Unary RPC:** The client sends a single request message to the server and receives a single response message in return. This type of method is suitable for applications that need single request-response exchanges.
- **Server streaming RPC:** The client sends a single request message to the server and the server then responds with a stream of response messages. This type of method allows for continuous data exchange between the client and server.
- **Client streaming RPC:** The client sends a stream request message to the server and the server then responds with a response message. Similar to server streaming RPC, this type of method also allows for a continuous data exchange but the data change is initiated by the client.
- **Bidirectional streaming RPC:** The client initiates the process by sending a stream request message, to which the server responds with a stream response message. This type of method enables communication between the client and the server to be conducted in both directions.

```
message CreateContactRequest {
    string first_name = 1;
    string last_name = 2;
    string email = 3;
    string phone = 4;
    int32 year_of_birth = 5;
    bool is_active = 6;
}

message CreateContactResponse {
    string contact_id = 1;
}

service ContactService {
    rpc CreateContact(CreateContactRequest) returns (CreateContactResponse);
}
```

```

using Grpc.Core;
using JLokaGrpcService.Protos;

namespace JLokaGrpcService.Services
{
    public class ContactGrpcService(ILogger<ContactGrpcService> logger):
    ContactService.ContactServiceBase
    {
        public override Task<CreateContactResponse>
        CreateContact(CreateContactRequest request, ServerCallContext context)
        {
            return Task.FromResult(new CreateContactResponse
            {
                ContactId = Guid.NewGuid().ToString(),
            });
        }
    }
}

-----

// Configure the HTTP request pipeline.
app.MapGrpcService<GreeterService>();
app.MapGrpcService<ContactGrpcService>();
app.MapGet("/", () => "Communication with gRPC endpoints must be made through a gRPC
client. To learn how to create a client, visit:
https://go.microsoft.com/fwlink/?linkid=2086909");

```

For creating grpc client, we need these modules:

```

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <!--we need these modules -->
  <ItemGroup>
    <PackageReference Include="Google.Protobuf" Version="3.23.1" />
    <PackageReference Include="Grpc.Net.Client" Version="2.57.0" />
    <PackageReference Include="Grpc.Tools" Version="2.57.0">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\JLokaGrpcService\JLokaGrpcService.csproj" />
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="Protos\greet.proto" GrpcServices="Client" />
    <Protobuf Include="Protos\jloka.proto" GrpcServices="Client" />
  </ItemGroup>
</Project>

```

And the invoice client class:

```
using Grpc.Net.Client;
using JLokaGrpcService.Protos;

namespace JLokaGrpcClient
{
    internal class InvoiceClient
    {
        public async Task CreateContactAsync()
        {
            using var channel = GrpcChannel.ForAddress("http://localhost:5077");
            // this is from launchSettings.json
            var client = new ContactService.ContactServiceClient(channel);
            var reply = await client.CreateContactAsync(new CreateContactRequest
            {
                Email = "john.doe@abc.com",
                FirstName = "John",
                LastName = "Doe",
                IsActive = true,
                Phone = "1234567890",
                YearOfBirth = 1980
            });

            Console.WriteLine("Created Contact: " + reply.ContactId);
            Console.ReadKey();
        }
    }
}
```

And to call it, in program.cs:

```
using JLokaGrpcClient;

Console.WriteLine("Hello, World!");
var contactClient = new InvoiceClient();
await contactClient.CreateContactAsync();
*****
```

Once the server **starts to send the stream response message**, the client **cannot send any more messages** to the server, unless the server finishes sending the stream response message or **the client cancels the RPC call by raising ServerCallContext.CancellationToken**.

```
*****
```

The server streaming service is useful when we need to send a stream of data to the client. In this case, the server can send multiple messages to the client over a single RPC call. Here are some scenarios where a server streaming service is useful:

- **Events streaming:** When the server needs to send a stream of event messages to the client so that the client can process the event messages.
- **Real-time data feeds:** When the server has a continuous stream of data to send to the client, such as stock prices, weather data, and so on.
- **File streaming:** When the server needs to send a large file to the client, the server can split the file into small chunks and send them one by one as a stream response message. This can reduce the memory usage on the server and the client because the server and the client do not need to load the entire file into memory.

To use stream from the server, we use: **stream-keyword** for response:

```
syntax = "proto3";
```

```
option csharp_namespace = "JLokaGrpcService.Protos";
```

```
message GetRandomNumbersRequest {
```

```
    int32 min = 1;
```

```
    int32 max = 2;
```

```
    int32 count = 3;
```

```
}
```

```
message GetRandomNumbersResponse {
```

```
    int32 number = 1;
```

```
}
```

```
service RandomNumbers {
```

```
    rpc GetRandomNumbers(GetRandomNumbersRequest) returns (stream  
    GetRandomNumbersResponse);
```

```
}
```

2. If we need a continuous stream response message, we can check the `ServerCallContext.CancellationToken` property of the context parameter. If the client cancels the RPC call, the `ServerCallContext.CancellationToken` property will be raised. The following code shows how to check the `ServerCallContext.CancellationToken` property:

```
public override async Task
GetRandomNumbers(GetRandomNumbersRequest request,
    IServerStreamWriter<GetRandomNumbersResponse>
responseStream, ServerCallContext context)
{
    var random = new Random();
    while (!context.CancellationToken.IsCancellationRequested)
    {
        await responseStream.WriteAsync(new
GetRandomNumbersResponse
        {
            Number = random.Next(request.Min, request.Max)
        });
        await Task.Delay(1000, context.CancellationToken);
    }
}
```

The streaming from server, and the client will handle it in this way:

```
using Grpc.Core;
using JLokaGrpcService.Protos;

namespace JLokaGrpcService.Services
{
    public class RandomNumbersService(ILogger<RandomNumbersService> _logger):
RandomNumbers.RandomNumbersBase
    {
        public override async Task GetRandomNumbers(GetRandomNumbersRequest request,
IServerStreamWriter<GetRandomNumbersResponse> responseStream,
            ServerCallContext context)
        {
            var random = new Random();
            for (var i = 0; i < request.Count; i++)
            {
                await responseStream.WriteAsync(new GetRandomNumbersResponse
                {
                    Number = random.Next(request.Min, request.Max)
                });
                await Task.Delay(1000);
            }
        }
    }
}
```

The client is:

```
using Grpc.Core;
using Grpc.Net.Client;
using JLokaGrpcService.Protos;

namespace JLokaGrpcClient
{
    internal class RandomClient
    {
        public async Task GetRandom()
        {
            var channel = GrpcChannel.ForAddress("https://localhost:7240");
            var client = new RandomNumbers.RandomNumbersClient(channel);

            var replay = client.GetRandomNumbers(new GetRandomNumbersRequest
            {
                Count = 10,
                Min = 1,
                Max = 100,
            });

            await foreach (var item in replay.ResponseStream.ReadAllAsync())
            {
                Console.WriteLine("The Number is: " + item.Number);
            }
            Console.ReadKey();
        }
    }
}
```

The streaming from client, and the server will handle it in this way:

```
using Grpc.Core;
using JLokaGrpcService.Protos;

namespace JLokaGrpcService.Services
{
    public class RandomNumbersClientService(ILogger<RandomNumbersClientService>
logger) : RandomNumbersClient.RandomNumbersClientBase
    {
        public override async Task<SendRandomNumbersResponse>
SendRandomNumbersClient(IAsyncStreamReader<SendRandomNumbersRequest>
requestStream, ServerCallContext context)
        {
            var count = 0;
            var sum = 0;
            await foreach (var request in requestStream.ReadAllAsync())
            {
                logger.LogInformation($"Received: {request.Number}");
                ++count;
            }
        }
    }
}
```



```

        sum += request.Number;
    }
    return new SendRandomNumbersResponse
    {
        Sum = sum,
        Count = count
    };
}
}
}

```

And the client will send the request in this way:

```

using Grpc.Net.Client;
using JLokaGrpcService.Protos;
namespace JLokaGrpcClient
{
    internal class ClientStreamClient
    {
        public async Task SendClientRandomNumbers()
        {
            var channel = GrpcChannel.ForAddress("https://localhost:7240");
            var client = new
RandomNumbersClient.RandomNumbersClientClient(channel);
            //create the streaming request
            using var clientStreamingCall = client.SendRandomNumbersClient();
            var random = new Random();
            for(var i=0; i<5; ++i)
            {
                await clientStreamingCall.RequestStream.WriteAsync(new
SendRandomNumbersRequest
                {
                    Number = random.Next(1, 100)
                });
                await Task.Delay(100);
            }
            // in this way we tell the server that the streaming end
            await clientStreamingCall.RequestStream.CompleteAsync();
            // Get the response
            var response = await clientStreamingCall;
            Console.WriteLine($"Count: {response.Count}, Sum: {response.Sum}");
            Console.ReadKey();
        }
    }
}

```

And the proto file is:

```
syntax = "proto3";

option csharp_namespace = "JLokaGrpcService.Protos";

message SendRandomNumbersRequest {
    int32 number = 1;
}

message SendRandomNumbersResponse {
    int32 count = 1;
    int32 sum = 2;
}

service RandomNumbersClient {
    rpc SendRandomNumbersClient(stream SendRandomNumbersRequest) returns
(SendRandomNumbersResponse);
}

*****
```

The bidirectional proto file:

```
message ChatMessage {
    string sender = 1;
    string message = 2;
}

service Chat {
    rpc SendMessage(stream ChatMessage) returns (stream ChatMessage);
}
```

The service from the server will be:

```
using Grpc.Core;
using JLokaGrpcService.Protos;

namespace JLokaGrpcService.Services
{
    public class ChatService(ILogger<ChatService> logger): Chat.ChatBase
    {
        public override async Task SendMessage(IAsyncStreamReader<ChatMessage>
requestStream, IServerStreamWriter<ChatMessage> responseStream,
ServerCallContext context)
        {
            await foreach (var request in requestStream.ReadAllAsync())
            {
                logger.LogInformation($"Received: {request.Message}");
                logger.LogInformation($"Received: {request.Sender}");
            }
        }
    }
}
```

```

        await responseStream.WriteAsync(new ChatMessage
        {
            Message = $"You said: {request.Message.ToUpper()}",
        });
    }
}
}
}
}

```

And the client will handle it (with background task for response):

```

using Grpc.Core;
using Grpc.Net.Client;
using JLokaGrpcService.Protos;

namespace JLokaGrpcClient
{
    internal class BiStreamChat
    {
        public async Task SendMessage()
        {
            using var channel = GrpcChannel.ForAddress("https://localhost:7240");
            var client = new Chat.ChatClient(channel);
            // Create streaming request
            using var streamingCall = client.SendMessage();
            Console.WriteLine("Start background task to receive messages...");
            var responseReaderTask = Task.Run(async () =>
            {
                await foreach (var response in
streamingCall.ResponseStream.ReadAllAsync())
                {
                    Console.WriteLine($"Replay from server: {response.Message}");
                }
            });
            Console.WriteLine("Starting to send messages...");
            while(true)
            {
                Console.Write("Your Message + enter: ");
                var message = Console.ReadLine();
                if (string.IsNullOrEmpty(message))
                {
                    break;
                }
                await streamingCall.RequestStream.WriteAsync(new ChatMessage
                {
                    Message = message,

```

```
        });  
    }  
  
    Console.WriteLine("Disconnecting...");  
    await streamingCall.RequestStream.CompleteAsync();  
    await responseReaderTask;  
}  
}  
}
```